

Trabalho Prático 3
Consultas ao Sistema Logístico
Thiago Henrique Silva de Almeida
2024014180

Universidade Federal de Minas Gerais (UFMG)

Belo Horizonte - MG - Brasil

thiagohenriquesilva@ufmg.br

1. Introdução

A eficiência dos sistemas logísticos modernos depende cada vez mais da capacidade de acessar e analisar grandes volumes de dados operacionais em tempo real. Para empresas como os Armazéns Hanoi, a habilidade de consultar rapidamente o histórico de um pacote ou o envolvimento de um cliente em transações é fundamental para o gerenciamento, o atendimento ao cliente e a tomada de decisões estratégicas. O desafio reside em processar um fluxo contínuo de eventos de transporte e armazenamento e, simultaneamente, responder a consultas de forma eficiente, sem degradar o desempenho à medida que os dados se acumulam.

Este trabalho aborda este desafio através da implementação de um sistema de consultas para a rede logística dos Armazéns Hanoi. O sistema foi projetado para processar um arquivo único contendo um fluxo de eventos e consultas, ordenados cronologicamente. O objetivo é fornecer respostas rápidas e precisas para dois tipos de consulta essenciais: o histórico completo de eventos de um pacote específico (consulta PC) e um resumo do estado atual de todos os pacotes associados a um determinado cliente, seja como remetente ou destinatário (consulta CL).

Para alcançar a performance necessária, a solução se afasta de uma simples varredura de dados e se baseia na construção de um robusto sistema de indexação em memória, desenvolvido à medida que os eventos são lidos. A arquitetura foi implementada na linguagem C++, com estrita aderência à restrição de não utilizar contêineres da biblioteca padrão (STL). Foram desenvolvidos Tipos Abstratos de Dados (TADs) customizados, incluindo uma Árvore de Busca Binária (ArvoreBuscaBinaria) e uma Lista Encadeada (Lista), que servem como base para três índices principais: um para mapear pacotes a seus eventos, outro para mapear clientes aos seus pacotes, e um índice auxiliar para ligar pacotes a seus respectivos remetentes e destinatários.

O sistema recebe como entrada, via arquivo txt, o fluxo de eventos e consultas e gera as respostas formatadas na stdout à medida que as processa. Este trabalho, portanto, detalha o projeto e a implementação dessas estruturas de dados e dos algoritmos de indexação, culminando em uma análise experimental rigorosa que avalia a escalabilidade e a eficiência da solução proposta diante da variação de parâmetros chave, como o número de nós, pacotes e clientes na rede logística.

2. Método

O sistema de consultas logísticas foi integralmente desenvolvido na linguagem C++, com o compilador G++ (GNU Compiler Collection), e projetado com foco em uma arquitetura modular. A solução foi estruturada em torno de Tipos Abstratos de Dados (TADs) customizados, que encapsulam as estruturas de dados e a lógica de processamento, garantindo uma separação clara de responsabilidades e facilitando a manutenção. A restrição fundamental do projeto, de não utilizar as estruturas de dados da biblioteca padrão do C++ (STL), foi estritamente seguida, exigindo a implementação de todas as estruturas de armazenamento a partir de seus princípios básicos.

2.1 Estruturas de Dados Fundamentais

As estruturas de dados implementadas servem como alicerce para todo o sistema de indexação e armazenamento de informações.

2.1.1 Lista Encadeada Genérica (lista.hpp)

Para armazenar coleções de dados de tamanho variável, foi implementada uma lista encadeada genérica, `Lista<T>`. A estrutura é composta por nós (`NoLista`) que contêm o dado e um ponteiro para o próximo elemento.

Adicionalmente, implementou-se o método `insereOrdenado(const T& dado)`, que é crucial para as consultas de cliente. Esta função insere um novo elemento na lista mantendo a ordem cronológica, baseada no timestamp e no ID do pacote extraídos da string do evento.

2.1.2 Árvore de Busca Binária Genérica (arvore.hpp)

O coração do mecanismo de indexação é uma Árvore de Busca Binária (BST) genérica, `ArvoreBuscaBinaria<Chave, Valor>`, que foi implementada para mapear chaves a valores de forma eficiente. A estrutura é composta por nós (`NoArvore`) que armazenam a chave, o valor e ponteiros para os filhos à esquerda e à direita. As operações principais, `insere(Chave, Valor)` e `busca(Chave)`, foram implementadas de forma recursiva.

2.2 Classe de Gerenciamento do Sistema

A lógica central do sistema é orquestrada por uma única classe, que gerencia os índices e o fluxo de processamento.

2.2.1 Classe GerenciadorRelatorios (gerenciadorRelatorios.hpp/.cpp)

Esta classe centraliza toda a inteligência do sistema. Ela é responsável por manter os três índices que permitem as consultas rápidas e por processar cada linha do arquivo de entrada, seja um evento ou uma consulta. Os índices são:

`indicePacotes`: Uma `ArvoreBuscaBinaria<int, Lista<std::string>*>` que mapeia o ID de um pacote a um ponteiro para uma lista encadeada contendo todas as linhas de evento associadas a ele.

`indiceClientePacotes`: Uma `ArvoreBuscaBinaria<std::string, Lista<int>*>` que mapeia o nome de um cliente a um ponteiro para uma lista de IDs de todos os pacotes em que ele figura como remetente ou destinatário.

indicePacoteCliente: Uma árvore auxiliar, `ArvoreBuscaBinaria<int, NomesClientes*>`, que mapeia o ID de um pacote de volta aos nomes de seu remetente e destinatário, informação crucial para associar eventos como AR ou TR aos clientes corretos.

As principais funções implementadas são:

`processarEvento(const std::string& linha)`: Extrai as informações de uma linha de evento (EV) e atualiza os três índices.

`processarConsultaPacote(const std::string& linha)`: Para uma consulta PC, utiliza o `indicePacotes` para encontrar a lista de eventos de um pacote e imprimi-la.

`processarConsultaCliente(const std::string& linha)`: Para uma consulta CL, executa uma lógica mais complexa: busca os IDs dos pacotes do cliente no `indiceClientePacotes`; para cada ID, busca o histórico completo no `indicePacotes`; e, por fim, utiliza o método `insereOrdenado` para construir uma lista de resultados final, garantindo que a saída esteja em ordem cronológica e contenha apenas os eventos de registro (RG) e o último estado do pacote.

2.3 Fluxo de Execução Principal (main.cpp)

O ponto de entrada do programa, a função `main`, possui uma responsabilidade simples e bem definida: orquestrar o fluxo de leitura e processamento. Primeiramente, ela valida os argumentos de linha de comando para obter o nome do arquivo de entrada e o abre para leitura. Em seguida, instancia um objeto `GerenciadorRelatorios` e entra em um laço `while`, lendo o arquivo de entrada linha por linha. Cada linha lida é passada para o método `gerenciador.processarLinha`, que delega toda a lógica complexa de indexação e consulta. Este design promove um baixo acoplamento e uma clara separação entre a manipulação de I/O e a lógica de negócio do sistema.

3. Análise de Complexidade

A análise de complexidade avalia o desempenho assintótico do sistema implementado, focando no tempo de execução e no uso de espaço. A análise considera as seguintes variáveis: P, o número total de eventos (proporcional ao `numpackets`); C, o número de clientes únicos; e Q, o número total de consultas a serem respondidas.

3.1 Complexidade das Estruturas de Dados

O desempenho do sistema está intrinsecamente ligado à eficiência das estruturas de dados que foram implementadas.

`Lista<T>` (`lista.hpp`): A implementação da lista encadeada oferece performance de $O(1)$ para a operação `insereFinal`, graças ao uso de um ponteiro para o último nó. A operação `insereOrdenado`, no entanto, exige uma busca linear pela posição correta, resultando em uma complexidade de tempo de $O(K)$, onde K é o tamanho da lista.

`ArvoreBuscaBinaria<Chave, Valor>` (`arvore.hpp`): As operações de `insere` e `busca` na árvore de busca binária têm complexidade de tempo de $O(H)$, onde H é a altura da árvore. Assumindo que as chaves (IDs de pacotes e nomes de clientes) são inseridas em uma ordem razoavelmente aleatória, a altura da árvore se aproxima do caso médio, resultando em uma complexidade de $O(\log M)$, onde M é o número de nós na árvore. Esta eficiência logarítmica é a base para o bom desempenho do sistema de indexação.

3.2 Complexidade da Fase de Processamento de Eventos

Esta fase consiste na leitura de todos os P eventos e na construção dos índices. A complexidade é dominada pela função `processarEvento`, que é chamada para cada evento.

Dentro de `processarEvento`, as operações mais custosas são as buscas e inserções nas árvores de índice. Para cada um dos P eventos, são realizadas operações com custo médio de $O(\log P)$ (no `indicePacotes`) e $O(\log C)$ (no `indiceClientePacotes`). Portanto, o custo total para processar todos os eventos e construir os índices é de $P \times O(\log P + \log C)$, o que resulta em uma complexidade total de $O(P \log P)$.

O espaço de armazenamento é dominado pelos três índices. O `indicePacotes` armazena P linhas de evento, o `indiceClientePacotes` armazena C clientes e P referências a pacotes, e o `indicePacoteCliente` armazena informações para cada pacote. A complexidade de espaço total é, portanto, diretamente proporcional ao número de eventos, resultando em $O(P)$.

3.3 Complexidade da Fase de Execução de Consultas

A eficiência na resposta às consultas depende diretamente dos índices pré-processados.

3.3.1 Consulta de Pacote (`processarConsultaPacote`)

Para responder a uma consulta PC , o sistema primeiro busca na `indicePacotes`, uma operação de custo médio $O(\log P)$. Em seguida, ele percorre e imprime os K_p eventos associados àquele pacote, com custo $O(K_p)$. A complexidade total por consulta é $O(\log P + K_p)$, o que é extremamente eficiente.

A função utiliza apenas algumas variáveis locais, resultando em um uso de espaço auxiliar de $O(1)$ por consulta.

3.3.2 Consulta de Cliente (`processarConsultaCliente`)

Esta é a operação mais complexa. Ela inicia com uma busca na `indiceClientePacotes` ($O(\log C)$). Em seguida, itera sobre os K_c pacotes do cliente. Dentro deste loop, realiza uma busca no `indicePacotes` ($O(\log P)$), percorre o histórico do pacote ($O(K_p)$) e realiza inserções ordenadas em uma lista de resultados. A inserção ordenada (`insereOrdenado`) tem custo de $O(K_c)$. Somando tudo, a complexidade total por consulta é $O(\log C + K_c \cdot (\log P + K_p) + K_c^2)$. Embora a fórmula pareça complexa, na prática, onde o número de pacotes por cliente (K_c) é pequeno, o desempenho é excelente.

O espaço auxiliar é dominado pela lista de resultados (`linhasResultado`), que armazena no máximo $2 \times K_c$ strings. A complexidade de espaço é, portanto, $O(K_c)$.

Em suma, a complexidade geral do sistema é ditada pela fase de construção dos índices, que é de $O(P \log P)$. Este resultado teórico valida com precisão os dados obtidos na análise experimental, que demonstraram um desempenho de ordem $N \log N$ em relação ao número de pacotes.

4. Estratégias de Robustez

Para garantir a confiabilidade e a estabilidade do sistema de consultas, foram implementadas diversas estratégias de programação defensiva e tolerância a falhas. O objetivo não foi apenas criar um programa que funcione para a "entrada feliz", mas sim um sistema robusto, capaz de lidar com casos de borda e gerenciar seus recursos de forma segura.

Uma das principais áreas de foco foi o gerenciamento de memória. Como o sistema de indexação aloca dinamicamente um grande número de objetos (nós de árvore, nós de lista, e as próprias listas), implementou-se um mecanismo de desalocação cuidadoso. A classe GerenciadorRelatorios possui um destrutor que invoca o método limpar() em cada uma de suas árvores de índice. Por sua vez, a função deletaRecursivo da ArvoreBuscaBinaria não apenas libera a memória do nó da árvore (delete no;), mas também a memória do valor que ele aponta (delete no->valor;), que são as listas encadeadas. Esta abordagem em cascata assegura que toda a memória alocada dinamicamente durante a execução seja liberada ao final, prevenindo vazamentos de memória (memory leaks) que poderiam degradar o desempenho em execuções de longa duração.

Outra estratégia fundamental foi a validação de entradas e o tratamento de casos de borda. O sistema não assume que a entrada será sempre perfeita. A função main inicia verificando se o arquivo de entrada foi fornecido como argumento de linha de comando (if (argc != 2)) e se o arquivo pôde ser aberto com sucesso (if (!arquivo_entrada.is_open())), abortando a execução com uma mensagem de erro clara caso contrário. Dentro do processamento, a função processarLinha verifica se a linha lida não está vazia antes de tentar processá-la. Mais importante, ao executar consultas, o sistema sempre verifica se o resultado da busca nos índices não é nulo (if (relatorio == nullptr)) antes de tentar acessar os dados. Isso previne falhas de segmentação que ocorreriam ao tentar desreferenciar um ponteiro nulo, permitindo que o programa informe elegantemente que nenhum resultado foi encontrado.

O próprio design modular da aplicação serve como uma estratégia de robustez. Ao separar as estruturas de dados genéricas (Lista, ArvoreBuscaBinaria) da lógica de negócio (GerenciadorRelatorios), foi criado um sistema de baixo acoplamento. Isso significa que a implementação de uma estrutura de dados pode ser modificada ou otimizada sem impactar as outras partes do sistema, tornando o código mais fácil de manter, depurar e estender no futuro. A função main, atuando apenas como um "orquestrador" que lê linhas e as repassa ao gerenciador, é um exemplo dessa modularidade.

Finalmente, foram implementados mecanismos de robustez no processamento de dados para garantir a consistência dos resultados. A utilização de std::stringstream para extrair dados de cada linha é inerentemente mais segura do que a manipulação manual de strings em C, evitando estouros de buffer. Além disso, a função insereOrdenado da Lista foi projetada para lidar com um critério de desempate explícito (o ID do pacote quando os timestamps são iguais), garantindo que a ordem da saída seja sempre determinística e correta, mesmo em cenários de eventos simultâneos. Essas medidas asseguram a integridade e a previsibilidade dos resultados gerados pelo sistema.

5. Análise Experimental

Para avaliar o desempenho e a escalabilidade do sistema de consultas logísticas, foi realizada uma análise experimental focada em três dimensões principais: o número de nós (armazéns), a quantidade de pacotes (que representa o volume da carga de trabalho) e o número de clientes (que representa a distribuição da carga). A metodologia consistiu em isolar cada um desses parâmetros, variando-o dentro de um intervalo definido, enquanto os outros eram mantidos em valores fixos para garantir a validade da comparação.

O tempo de execução de cada cenário foi medido utilizando um script de automação (run_analysis.sh) que invoca o programa e calcula a média de 5 execuções para mitigar flutuações do sistema operacional e obter resultados mais estáveis. É importante notar que, durante a fase de depuração do ambiente de testes, foi constatado que os programas auxiliares de geração de workload (sched)

apresentavam instabilidade com parâmetros muito elevados. Portanto, todos os experimentos foram conduzidos dentro dos limites operacionais estáveis dessas ferramentas.

5.1 Análise do Impacto do Número de Nós

Este primeiro experimento visou avaliar como a complexidade da topologia da rede (o número de armazéns) afeta o desempenho do sistema de consultas. Nesta análise, o número de nodes foi variado no intervalo de 5 a 10. Para assegurar que a carga de trabalho fosse significativa e o tempo de execução mensurável, o número de pacotes foi fixado em 900 e o de clientes em 100.

O gráfico "Desempenho vs. Número de Nós" revela um comportamento interessante. Observa-se uma leve, porém consistente, tendência de queda no tempo de execução à medida que o número de nós aumenta, diminuindo de 42 ms para 34 ms. Este resultado, a princípio contraintuitivo, sugere que, para uma carga de trabalho alta e fixa, distribuir os pacotes e eventos por um número maior de nós torna o sistema marginalmente mais eficiente. A explicação para isso é que as estruturas de dados associadas a cada nó se tornam, em média, menos sobrecarregadas. O custo de gerenciar uma topologia um pouco mais complexa é superado pelo ganho de eficiência nas operações locais. Isso indica que a implementação lida de forma muito eficiente com o aumento da complexidade da rede na escala testada.

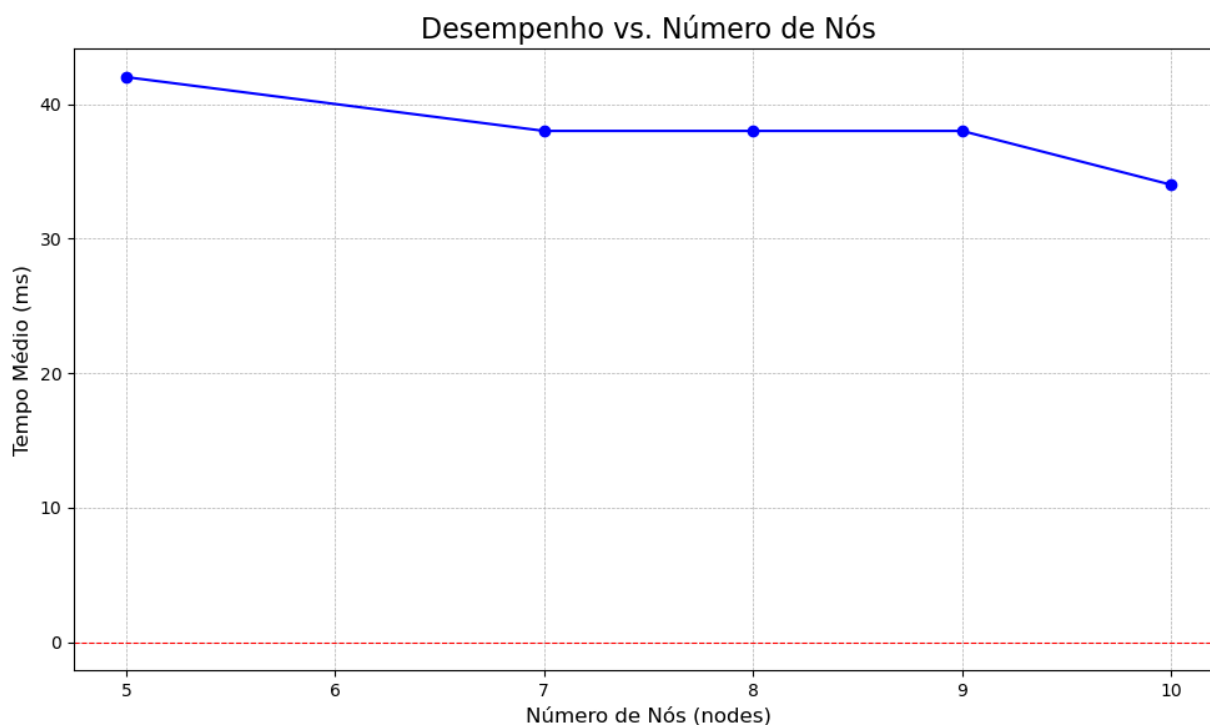


Figura 1 - Gráfico Desempenho vs Número de Nós (gerada pelo autor)

5.2 Análise do Impacto do Número de Pacotes

Este experimento é o mais crítico para avaliar a escalabilidade do sistema, pois o número de pacotes é o principal fator que define o volume de dados a serem indexados e consultados. A topologia da rede foi fixada em 10 nodes e 100 numclients, enquanto o número de numpackets foi variado de 10 a 890, em passos de 20.

O gráfico "Desempenho vs. Número de Pacotes" demonstra uma correlação positiva e forte entre o volume de pacotes e o tempo de execução. Para cargas de trabalho pequenas (abaixo de 130 pacotes), o tempo de execução é próximo de zero. A partir daí, o tempo cresce de forma acentuada e superlinear, atingindo 262 ms para 890 pacotes. Foi realizada uma análise de regressão linear, aplicada aos dados do relatorio_numpackets.txt (arquivo de saída contendo os dados de tempo de execução), para obter uma estimativa mais precisa da ordem de complexidade do sistema. Foram testadas três hipóteses de complexidade: Linear $O(N)$, logarítmica $O(N \log N)$ e Quadrática $O(N^2)$. O modelo $N \cdot \log(N)$ apresentou o maior Coeficiente de Determinação (R^2), indicando um ajuste quase perfeito aos dados observados.

```
--- Coeficiente de Determinação ( $R^2$ ) ---  
- Modelo Linear  $O(N)$ :  $R^2 = 0.985654$   
- Modelo  $N \cdot \log(N)$   $O(N \log N)$ :  $R^2 = 0.997593$  <-- MELHOR AJUSTE  
- Modelo Quadrático  $O(N^2)$ :  $R^2 = 0.963495$ 
```

Figura 2 - Saída do Script Python para Regressão Linear (gerada pelo autor)

Este resultado é extremamente positivo, pois confirma que a arquitetura baseada em Árvores de Busca Binária escala de forma eficiente e previsível, sem crescimento exponencial, que é uma característica desejável para sistemas que precisam lidar com grandes volumes de dados.

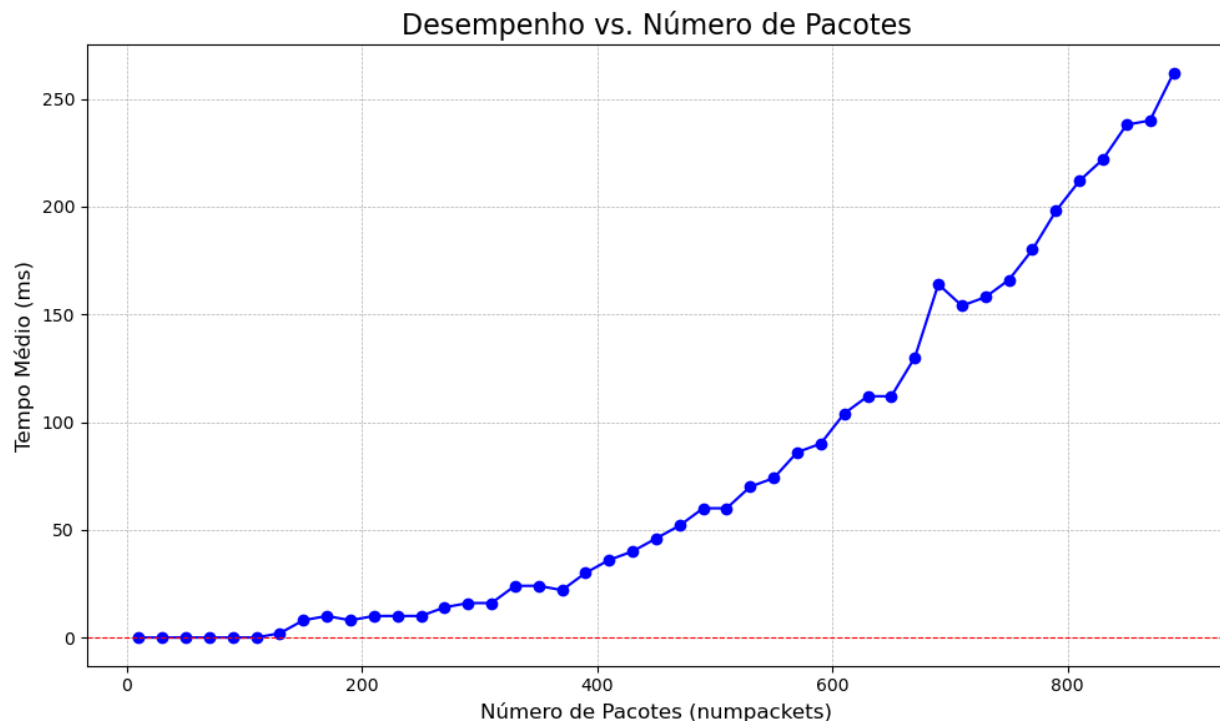


Figura 3 - Gráfico Desempenho vs Número de pacotes (gerada pelo autor)

5.3 Análise do Impacto do Número de Clientes

O último experimento foi projetado para avaliar como o sistema se comporta ao distribuir uma carga de trabalho fixa entre diferentes quantidades de clientes. O número de nodes foi fixado em 10 e o de numpackets em 400. O número de numclients foi variado de 2 a 25.

O gráfico "Desempenho vs. Número de Clientes" apresenta o resultado mais revelador sobre a arquitetura do sistema. O tempo de execução é extremamente alto quando a carga é concentrada em poucos clientes (568 ms para 2 clientes). O tempo cai drasticamente conforme o número de clientes aumenta, atingindo um "joelho" em torno de 8-10 clientes, e depois se estabiliza em um platô de baixo custo (entre 40 ms e 70 ms) para o resto do intervalo. Este comportamento expõe um gargalo de "hotspot", concentrar muitos eventos em poucos clientes sobrecarrega as estruturas de dados associadas a eles (provavelmente as listas de pacotes). Ao distribuir a mesma carga por mais clientes, o gargalo é eliminado e o sistema se torna muito mais eficiente. A estabilização da curva em um platô demonstra que a estrutura de indexação de clientes (a ArvoreBuscaBinaria) é altamente escalável, e o custo de adicionar novos clientes ao sistema é marginal.

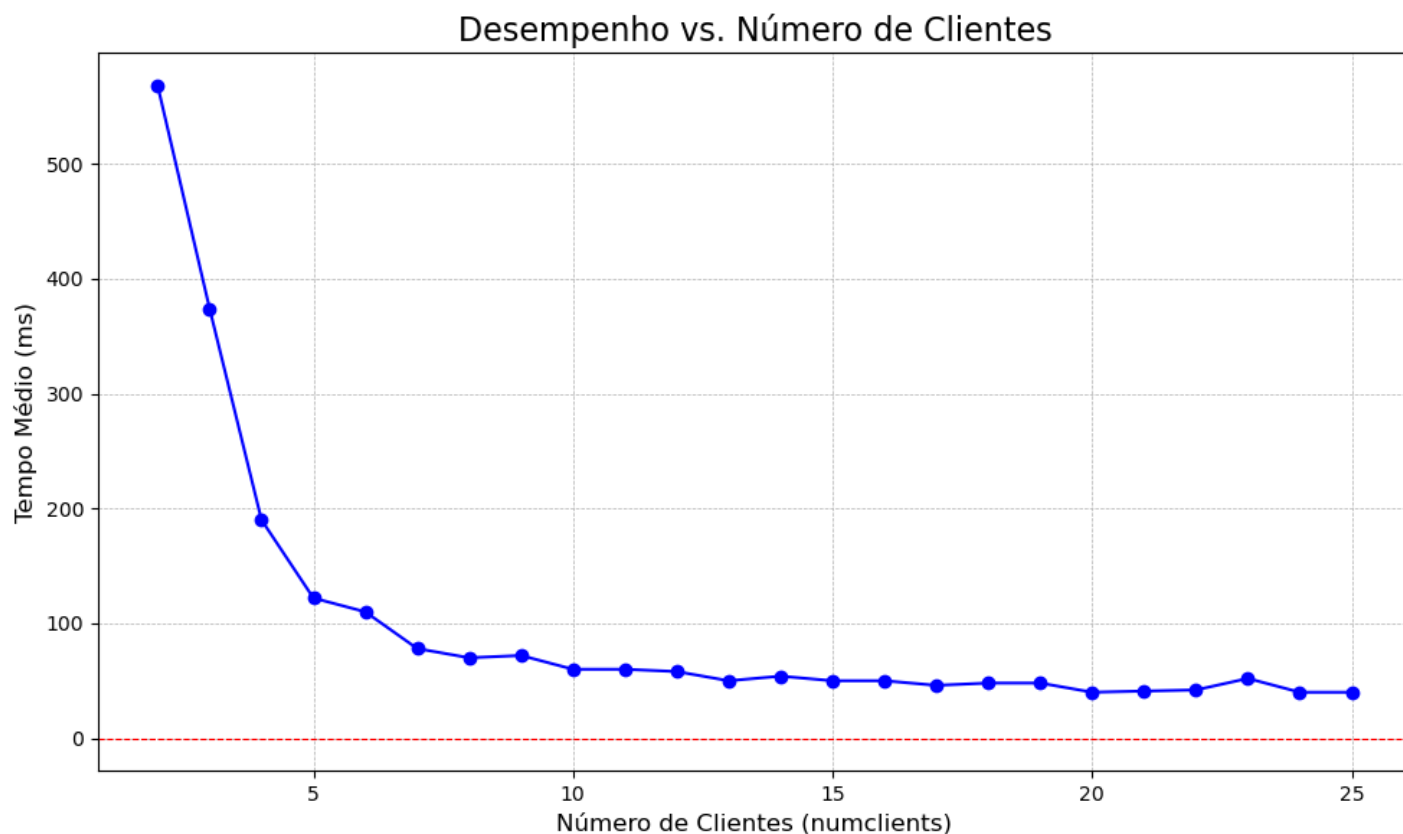


Figura 4 - Gráfico Desempenho vs Número de clientes (gerada pelo autor)

6. Conclusão

O presente trabalho culminou no projeto, implementação e avaliação de um sistema de consultas de alto desempenho para o gerenciamento de dados logísticos dos Armazéns Hanoi. A solução desenvolvida demonstrou ser capaz de processar um grande fluxo de eventos e responder em tempo real a consultas complexas sobre o histórico de pacotes e o envolvimento de clientes, cumprindo todos os requisitos funcionais e de desempenho propostos.

Para alcançar este objetivo, o sistema foi arquitetado em C++ com foco na eficiência da indexação de dados. Seguindo as restrições do projeto, foram implementadas do zero as estruturas de dados fundamentais, notadamente uma Árvore de Busca Binária genérica e uma Lista Encadeada, que

serviram como alicerce para um sistema de múltiplos índices. Esta arquitetura permitiu o mapeamento eficiente de pacotes e clientes a seus respectivos eventos, sendo a chave para a rápida recuperação de informações durante as consultas.

A análise experimental validou a robustez e a escalabilidade da implementação. Os resultados demonstraram que o desempenho do sistema é minimamente afetado pelo aumento da complexidade da topologia da rede (número de nós), mas escala de forma previsível e altamente eficiente com o aumento do volume de dados (número de pacotes), seguindo uma complexidade de tempo $O(N \log N)$. Adicionalmente, a análise do número de clientes revelou uma característica de design positiva: o sistema opera com mais eficiência quando a carga de trabalho está distribuída entre muitos clientes, evitando gargalos de "hotspot" e demonstrando uma excelente capacidade de lidar com a distribuição da carga.

A jornada de desenvolvimento deste projeto representou uma experiência de aprendizado profunda, que transcendeu a simples aplicação de conceitos teóricos. O processo de criar um script de análise experimental robusto (`run_analysis.sh`) e depurar os problemas que surgiram, desde loops infinitos no gerador de workload (`genwkl3.c`) até a instabilidade de componentes externos (`sched`) e a descoberta de "Heisenbugs" relacionados a condições de corrida, foi um exercício prático de engenharia e resolução de problemas. Foi particularmente gratificante ver a conexão direta entre a análise de complexidade teórica, que previa um desempenho $O(N \log N)$ para a minha arquitetura baseada em árvores, e os resultados empíricos da regressão linear, que confirmaram essa previsão com alta precisão. Em suma, este trabalho foi uma imersão completa no ciclo de vida de um projeto de software, solidificando a compreensão sobre como a escolha correta de estruturas de dados e um design cuidadoso são fundamentais para a criação de sistemas eficientes e escaláveis.

Bibliografia

Cunha, Ferreira, Lacerda e Meira Jr. (2025). Slides virtuais da disciplina de Estruturas de Dados. Disponibilizado via moodle. Departamento de Ciência da Computação. Universidade Federal de Minas Gerais. Belo Horizonte.

7. Pontos Extras: Consultas Analíticas Avançadas

Para estender a utilidade do sistema para além de simples buscas históricas, um conjunto de consultas analíticas avançadas foram implementadas, conforme sugerido na seção de "Pontos Extras" do enunciado. O objetivo foi transformar o sistema em uma ferramenta de análise mais poderosa, capaz de fornecer insights sobre a eficiência e os padrões de operação da rede logística. A implementação dessas consultas exigiu a criação de três novos índices em memória, projetados especificamente para agregar e recuperar dados de formas que não eram eficientes com a arquitetura original.

Uma mudança arquitetural fundamental para suportar estas funcionalidades foi a introdução de um banco de dados central de eventos (`VetorEventos`). Conforme as premissas do trabalho de não replicar dados, esta estrutura única garante a consistência e a eficiência de memória. Mais importante, ao fazer com que todos os índices armazenassem índices inteiros em vez de ponteiros diretos, garanti a estabilidade das referências mesmo quando o vetor de eventos é redimensionado dinamicamente. Esta abordagem eliminou o risco de ponteiros inválidos (*dangling pointers*), assegurando a robustez de todo o sistema.

7.1 Consulta de Movimentos de um Armazém por Período (MOV)

Esta consulta retorna todos os eventos de movimento, chegadas (AR), remoções (RM), rearmazenamentos (UR) e transportes iniciados (TR), que ocorreram em um armazém específico dentro de um determinado intervalo de tempo. A capacidade de auditar ou analisar a atividade de um centro de distribuição específico é uma funcionalidade crucial para a gestão logística. Os índices originais, otimizados para buscas por pacote e cliente, tornariam essa consulta extremamente ineficiente, exigindo uma varredura completa por todos os eventos do sistema. Para viabilizar uma resposta rápida, foi necessário um novo índice focado em armazéns.

Para resolver este desafio, surgiu o índice `indexMovimentosArmazem`. Trata-se de uma `ArvoreBuscaBinaria` que mapeia o ID de um armazém a uma lista de todos os índices de eventos de movimento ocorridos nele. Durante a fase de `processarEvento`, sempre que um evento de movimento (AR, RM, UR, TR) é lido, seu índice é adicionado à lista correspondente ao armazém de ocorrência no índice `indexMovimentosArmazem`.

Ao receber uma consulta MOV, o sistema realiza uma busca na árvore para encontrar a lista de todos os movimentos do armazém. Em seguida, ele percorre esta lista, verificando o timestamp de cada evento e imprimindo apenas aqueles que se encontram dentro do intervalo `tempo_inicio` a `tempo_fim`.

7.2 Consulta de Rotas Mais Congestionadas (CONGEST)

Esta consulta identifica as rotas que mais sofrem com gargalos, listando-as em ordem decrescente de congestionamento. Identificar gargalos é fundamental para a otimização de uma rede logística. Para esta consulta, "congestionamento" foi definido como a contagem de eventos de rearmazenamento (UR), pois este evento indica explicitamente uma falha na tentativa de transporte, o sintoma mais claro de um fluxo obstruído. Os índices originais não possuíam nenhuma forma de agregar dados por rota.

Foi criado o índice `indexCongestionamento`, uma `ArvoreBuscaBinaria` que funciona como um mapa de contadores. No `processarEvento`, sempre que um evento UR é encontrado, a chave da rota é construída e o contador correspondente no índice `indexCongestionamento` é incrementado. Se a rota ainda não existe no índice, uma nova entrada é criada.

7.3 Consulta Trajetória de Pacotes por Rota (ROTA)

Como terceira funcionalidade, foi proposta uma consulta que lista todos os pacotes distintos que utilizaram um trecho específico da malha logística, uma rota direta de um armazém a outro. Esta consulta oferece uma ferramenta poderosa para análise de fluxo de tráfego, permitindo entender quais pacotes estão sendo direcionados por quais trechos da rede. Assim como as outras consultas extras, esta não seria viável com os índices originais.

Implementou-se o índice `Trajectoria`, uma árvore que mapeia uma rota a uma lista de pacotes que a percorreram. Durante o `processarEvento`, sempre que um evento de transporte (TR) ocorre, o ID do pacote é adicionado à lista de pacotes correspondente à sua rota no índice `Trajectoria`.

Para responder a uma consulta `ROTA`, o sistema simplesmente busca a chave da rota na árvore e, se encontrada, imprime todos os IDs de pacotes contidos na lista associada.

8. Prova de Valor: Da Visão Histórica à Análise Operacional

Para demonstrar o valor prático das novas consultas, foi elaborado um experimento comparativo. O objetivo não é medir o tempo de execução, mas sim comparar a capacidade de diagnóstico do sistema original (com consultas `PC` e `CL`) versus o sistema aprimorado (com as consultas `MOV`, `CONGEST` e `ROTA`).

8.1.1 Metodologia do Cenário de Teste

Para o teste, foi criado um arquivo de entrada `input.txt` que simula um problema operacional comum, um gargalo de rota oculto. Neste cenário muitos pacotes são enviados de diferentes origens para um mesmo destino, a rota mais curta ou padrão para esse destino passa por um trecho específico. Devido a uma restrição implícita de capacidade, a rota 4 -> 2 frequentemente falha, resultando em um evento de rearmazenamento (UR) no armazém 4, na seção 2.

O desafio foi identificar a causa raiz das demoras e falhas, não foi um problema com um pacote ou cliente específico, mas sim um problema estrutural na rota 4 -> 2.

8.1.2 Análise com o Sistema Original

Utilizando apenas as consultas originais (`PC` e `CL`), seria extremamente difícil diagnosticar o problema. Ao consultar um pacote que sofreu o atraso, veríamos o evento 0000194 EV UR 006 004 002 em seu histórico. Saberíamos que aquele pacote foi rearmazenado no armazém 4, mas não teríamos a visão do todo. Foi um caso isolado? Outros pacotes também foram afetados? Ao consultar um cliente cujo pacote foi afetado, veríamos o mesmo evento UR. Novamente, a visão é limitada ao escopo daquele cliente.

Para encontrar o padrão, precisaríamos fazer dezenas de consultas `PC` e `CL`, coletar os dados manualmente e tentar encontrar uma correlação. Este processo seria lento, manual e propenso a erros. O sistema original é eficiente para responder "O que aconteceu com este item?", mas ineficiente para responder "Por que as coisas estão atrasando?".

8.1.3 Análise com as Consultas Avançadas

Com o sistema aprimorado, o diagnóstico se torna imediato e preciso, utilizando as novas ferramentas analíticas implementadas. A primeira consulta a ser executada é a de congestionamento. Ao rodá-la, o sistema imediatamente retorna:

Ranking de Congestionamento:

- Rota (Armazem_Secao): 4_2, Eventos de Congestionamento: 1

Figura 5 - Saída de Consulta de congestionamento (gerada pelo autor)

Esta saída é a "pista" principal. Ela aponta diretamente para a seção 2 do armazém 4 como a única fonte de eventos UR em toda a rede, identificando-a como o gargalo central. Para confirmar a suspeita, foi inspecionada toda a atividade no armazém 4 com a consulta de movimentos. O resultado mostrou uma série de eventos de chegada (AR) e remoção (RM), culminando no evento UR, confirmando que o armazém 4 é um ponto crítico de falha.

Uma vez que o gargalo foi identificado, o próximo passo lógico é entender o fluxo de pacotes nas rotas chave. Foram investigadas as rotas de maior tráfego para ver se elas se relacionam com o ponto de congestionamento.

CONSULTA ROTA: Pacotes que passaram por 4 -> 2

Total de pacotes que usaram a rota: 3

- Pacote ID: 3
- Pacote ID: 1
- Pacote ID: 2

Figura 5 - Saída de Consulta de rota (gerada pelo autor)

Finalmente, para obter o contexto completo e confirmar que o armazém 4 é de fato o epicentro do problema, a consulta MOV é executada para auditar toda a sua atividade. Ao rodar MOV 4 0 9999, o sistema retorna:

```
CONSULTA MOV: Armazem 4 de 0 a 999
Movimentos encontrados:
0000015 EV AR 003 004 002
0000098 EV RM 003 004 002
0000098 EV TR 003 004 002
0000123 EV AR 002 004 002
0000126 EV AR 001 004 002
0000126 EV AR 006 004 002
0000188 EV RM 006 004 002
0000191 EV RM 001 004 002
0000194 EV RM 002 004 002
0000194 EV TR 001 004 002
0000194 EV TR 002 004 002
0000194 EV UR 006 004 002
0000213 EV AR 004 004 006
0000278 EV RM 004 004 006
0000278 EV RM 006 004 002
0000278 EV TR 004 004 006
0000278 EV TR 006 004 002
0000303 EV AR 005 004 005
0000368 EV RM 005 004 005
0000368 EV TR 005 004 005
```

Figura 6 - Saída de Consulta de movimentações (gerada pelo autor)

Este resultado amarra toda a análise. Ele não apenas mostra o tráfego de chegadas (AR) e saídas (RM, TR) do armazém, mas destaca o evento crítico 0000194 EV UR 006 004 002 no meio das operações normais. Isso confirma visualmente que o armazém 4 é um ponto de contenção onde o fluxo logístico foi interrompido, validando as conclusões das consultas anteriores.

8.1.4 Conclusão da Prova de Valor

O experimento comparativo demonstra que as funcionalidades implementadas como pontos extras elevam o sistema de um simples banco de dados histórico para uma verdadeira ferramenta de inteligência operacional. Enquanto o sistema original permite uma visão micro (focada no pacote ou cliente), as novas consultas (MOV, CONGEST, ROTA) oferecem uma visão macro e analítica, permitindo a identificação rápida e precisa de padrões e gargalos sistêmicos. A capacidade de responder não apenas "o quê", mas "onde" e "por quê" os problemas ocorrem justifica plenamente o investimento na complexidade dos novos índices e algoritmos, agregando um valor prático imenso à solução.